RETAILMART V3 · SQL

Window Functions Part 3 -- LAG & LEAD

— Siraj Sir

📱 WhatsApp Announcement

Window Functions Part 3 (LAG & LEAD)

Yesterday you built running totals and moving averages. Today you look SIDEWAYS -- at the previous row and the next row. LAG and LEAD let you compare any row to its neighbor without a self-join.

Part 1: LAG -- Looking at the Previous Row
Part 2: LEAD -- Looking at the Next Row
Part 3: Period-over-Period Patterns

Month-over-month growth, gap between orders, price change detection -- these are all LAG/LEAD problems. Every analyst dashboard that shows 'vs last month' or 'change from previous' uses these functions.

Prerequisites: earlier sessions (OVER, PARTITION BY, running totals). Today completes your window function toolkit.

TODAY'S AGENDA (2 Hours)

Section	Time	Topics
Part 1: LAG -- Looking at the Previous Row	2 Hours	LAG syntax and defaults, Accessing previous row values, Calculating differences between consecutive rows
Part 2: LEAD -- Looking at the Next Row		LEAD syntax and use cases, Forward-looking comparisons, Gap analysis and duration calculations
Part 3: Period-over-Period Patterns		Month-over-month growth formula, Combining LAG with PARTITION BY, Multi-period comparisons and anomaly detection

YOUR SQL JOURNEY

- ✓ SQL Intro
- ✓ Setup & RetailMart
- ✓ DDL & Data Types
- ✓ Constraints & DML
- ✓ Filtering & Sorting
- ✓ Scalar Functions
- ✓ Conditional Logic
- ✓ Aggregation
- ✓ Self Join & UNION
- ✓ Subqueries Part 1
- ✓ Subqueries & CTEs
- ✓ Database Concepts
- ✓ Review & Practice
- ✓ Window Funcs Part 1
- ✓ Aggregation & Frames
- 18 **LAG & LEAD** ← HERE

RECAP

What We Covered Yesterday

- Window aggregates: SUM, AVG, COUNT with OVER()
- Running totals by adding ORDER BY inside OVER()
- Window frames: ROWS BETWEEN for moving averages
- Reset patterns: running totals that restart at boundaries

STORY

The Monthly Revenue Report

Your CFO opens the monthly dashboard and asks: 'How much did revenue change from last month?' You have monthly totals. But to calculate the CHANGE, you need each month's revenue sitting next to the PREVIOUS month's revenue. Without LAG, you would need a self-join. With LAG, it is one line.

What LAG Does

LAG reaches back to a previous row within your window and pulls a value from it. The row order is defined by ORDER BY inside OVER().

DEFINITION

LAG(column, offset, default) OVER (PARTITION BY … ORDER BY …)

- column -- which column to pull from the previous row
- offset -- how many rows back (default = 1, i.e. the immediate previous row)
- default -- value to return when there is no previous row (default = NULL)

LAG Function

```
-- Basic LAG

 LAG(column) OVER (ORDER BY col)
   -- Gets previous row's value

-- With Offset

 LAG(column, 2) OVER (ORDER BY col)
   -- 2 rows back

-- With Default

 LAG(column, 1, 0) OVER (ORDER BY col)
   -- Returns 0 instead of NULL
```

LAG Function

```
-- Partitioned
 LAG(column) OVER (
   PARTITION BY grp ORDER BY col)
   -- Resets at each group
```

Example: Previous Month Revenue

Get each month's revenue alongside the previous month's revenue:

LAG for Previous Month Revenue

```
SELECT
  DATE_TRUNC('month', order_date)  AS order_month,
  SUM(net_total)                AS revenue,
  LAG(SUM(net_total))
    OVER (ORDER BY DATE_TRUNC('month', order_date))
                                   AS prev_month_revenue
FROM sales.orders
WHERE order_status = 'Delivered'
GROUP BY DATE_TRUNC('month', order_date)
ORDER BY order_month;
```

Result:

LAG Result

```
order_month  | revenue    | prev_month_revenue
-------------+------------+-------------------
2025-01-01   | 12500000   | NULL
2025-02-01   | 11800000   | 12500000
2025-03-01   | 14200000   | 11800000
```

The first row has NULL for prev_month_revenue because there is no row before January. This is where the third argument (default value) helps.

LAG with a Default Value

Replace NULL with 0 for the first row:

LAG with Default

```
SELECT
  DATE_TRUNC('month', order_date)  AS order_month,
  SUM(net_total)                AS revenue,
  LAG(SUM(net_total), 1, 0)
    OVER (ORDER BY DATE_TRUNC('month', order_date))
                                   AS prev_month_revenue
FROM sales.orders
WHERE order_status = 'Delivered'
GROUP BY DATE_TRUNC('month', order_date)
ORDER BY order_month;
```

LAG with PARTITION BY

When you partition, LAG looks back within each partition independently:

LAG Partitioned by Store

```
SELECT
  store_id,
  DATE_TRUNC('month', order_date) AS order_month,
  SUM(net_total)               AS revenue,
  LAG(SUM(net_total))
    OVER (PARTITION BY store_id
          ORDER BY DATE_TRUNC('month', order_date))
                                 AS prev_month_revenue
FROM sales.orders
WHERE order_status = 'Delivered'
GROUP BY store_id, DATE_TRUNC('month', order_date)
ORDER BY store_id, order_month;
```

Each store's first month gets NULL -- the partition resets LAG's memory.

LAG with Offset > 1

Look back 2 rows to compare with 'two months ago':

LAG with Offset 2

```
SELECT
  DATE_TRUNC('month', order_date) AS order_month,
  SUM(net_total)               AS revenue,
  LAG(SUM(net_total), 2)
    OVER (ORDER BY DATE_TRUNC('month', order_date))
                                  AS two_months_ago
FROM sales.orders
WHERE order_status = 'Delivered'
GROUP BY DATE_TRUNC('month', order_date)
ORDER BY order_month;
```

PRO TIP

PRO TIP

LAG(col, 1) = previous row. LAG(col, 2) = two rows back. LAG(col, 12) = same month last year if you have monthly data. The offset is just a row count within the ordered window.

Practice 1: Month-over-Month Revenue Growth

EASY

SCENARIO: The finance team needs a monthly revenue report showing the percentage change from the previous month.

TASK: Write a query that shows each month's delivered revenue, the previous month's revenue, and the month-over-month growth percentage. Round the growth to 2 decimal places.

HINT: Growth % = (current - previous) / previous * 100. Use NULLIF on the denominator to avoid division by zero for the first month.

Answer

✅ ANSWER

```
SELECT
  DATE_TRUNC('month', order_date)  AS order_month,
  SUM(net_total)                AS revenue,
  LAG(SUM(net_total))
    OVER (ORDER BY DATE_TRUNC('month', order_date))
                                   AS prev_revenue,
  ROUND(
    (SUM(net_total) - LAG(SUM(net_total))
      OVER (ORDER BY DATE_TRUNC('month', order_date)))
    / NULLIF(LAG(SUM(net_total))
      OVER (ORDER BY DATE_TRUNC('month', order_date)), 0)
    * 100, 2
  )                                AS mom_growth_pct
FROM sales.orders
WHERE order_status = 'Delivered'
GROUP BY DATE_TRUNC('month', order_date)
ORDER BY order_month;
```

Practice 2: Order Gap Analysis

MEDIUM

SCENARIO: The CRM team wants to identify customers who have long gaps between orders -- potential churn signals.

TASK: For each customer, calculate the number of days between each order and their previous order. Show customer_id, order_id, order_date, previous_order_date, and days_since_last_order. Only include customers who have placed at least 2 orders.

HINT: Use LAG on order_date, partitioned by customer_id. Then subtract dates to get the gap in days.

Answer

✅ ANSWER

```
SELECT
  cust_id,
  order_id,
  order_date,
  LAG(order_date)
    OVER (PARTITION BY cust_id
          ORDER BY order_date)     AS prev_order_date,
  order_date - LAG(order_date)
    OVER (PARTITION BY cust_id
          ORDER BY order_date)     AS days_since_last_order
FROM sales.orders
WHERE cust_id IN (
  SELECT cust_id
  FROM sales.orders
  GROUP BY cust_id
  HAVING COUNT(*) >= 2
)
ORDER BY cust_id, order_date;
```

STORY

The Web Analytics Question

The product manager asks: 'What is the average time between page views in a user session?' Each page_view has a timestamp. To calculate the duration between THIS view and the NEXT one, you need to peek at the next row. That is exactly what LEAD does.

What LEAD Does

LEAD is the mirror image of LAG. Instead of looking back, it looks forward to the next row.

DEFINITION

LEAD(column, offset, default) OVER (PARTITION BY … ORDER BY …)

- Same three arguments as LAG
- offset = 1 means the next row (default)
- offset = 2 means two rows ahead
- default fills in when there is no next row (last row in partition)

LEAD Function

```
-- Basic LEAD

 LEAD(column) OVER (ORDER BY col)
   -- Gets next row's value

-- With Offset

 LEAD(column, 2) OVER (ORDER BY col)
   -- 2 rows ahead

-- With Default

 LEAD(column, 1, 0) OVER (ORDER BY col)
   -- Returns 0 instead of NULL
```

LEAD Function

```
-- Partitioned
 LEAD(column) OVER (
   PARTITION BY grp ORDER BY col)
   -- Resets at each group
```

Example: Next Order Date

For each order, show when the customer's next order will be:

LEAD for Next Order

```
SELECT
  cust_id,
  order_id,
  order_date,
  LEAD(order_date)
    OVER (PARTITION BY cust_id
          ORDER BY order_date)    AS next_order_date,
  LEAD(order_date)
    OVER (PARTITION BY cust_id
          ORDER BY order_date)
    - order_date                  AS days_until_next_order
FROM sales.orders
ORDER BY cust_id, order_date;
```

The last order for each customer shows NULL for next_order_date -- there is no 'next' row.

Comparison

```
-- Same result, different approach:

-- Using LAG (current row looks BACK):
revenue - LAG(revenue) OVER (ORDER BY month)  AS change

-- Using LEAD (previous row looks FORWARD):
LEAD(revenue) OVER (ORDER BY month) - revenue AS change

-- Both calculate the same difference.
-- Convention: LAG is more common for "change from previous".
-- LEAD is more natural for "time until next event".
```

PRO TIP

PRO TIP

Use LAG for backward comparisons (growth from last period). Use LEAD for forward calculations (time until next event, days to next order). They are interchangeable, but choosing the right one makes your intent clear.

Example: Web Session Duration

Calculate time between consecutive page views for each customer:

Session Duration with LEAD

```
SELECT
  customer_id,
  view_timestamp,
  page_url,
  LEAD(view_timestamp)
    OVER (PARTITION BY customer_id
          ORDER BY view_timestamp)
                                  AS next_view_time,
  EXTRACT(EPOCH FROM (
    LEAD(view_timestamp)
      OVER (PARTITION BY customer_id
            ORDER BY view_timestamp)
    - view_timestamp
  )) / 60.0                      AS minutes_until_next_view
FROM web_events.page_views
ORDER BY customer_id, view_timestamp;
```

Practice 3: First vs Second Order Delta

MEDIUM

SCENARIO: The marketing team wants to know how quickly new customers place their second order after their first.

TASK: For each customer, find their first order date and their second order date. Calculate the number of days between them. Show only customers who have placed at least 2 orders. Order by the gap (largest first).

HINT: Use ROW_NUMBER to identify the first and second orders, then use LEAD on the first row to get the second order's date. Alternatively, filter with ROW_NUMBER in a CTE.

Answer

✅ ANSWER

```
WITH numbered AS (
  SELECT
    cust_id,
    order_date,
    ROW_NUMBER()
      OVER (PARTITION BY cust_id
            ORDER BY order_date)  AS order_num,
    LEAD(order_date)
      OVER (PARTITION BY cust_id
            ORDER BY order_date)  AS next_order_date
  FROM sales.orders
)
SELECT
  cust_id,
  order_date        AS first_order_date,
  next_order_date   AS second_order_date,
  next_order_date - order_date
                    AS days_to_second_order
FROM numbered
WHERE order_num = 1
  AND next_order_date IS NOT NULL
ORDER BY days_to_second_order DESC;
```

Practice 4: Web Session Duration

MEDIUM

SCENARIO: The UX team needs to understand how long users spend browsing between pages.

TASK: For each page view, calculate the time in minutes until the customer's next page view. Then find the average time-between-views per customer. Only include gaps under 30 minutes (longer gaps are probably new sessions).

HINT: Use LEAD on event_timestamp partitioned by customer_id. Extract epoch to convert interval to seconds, then divide by 60 for minutes.

Answer (1/2)

✅ ANSWER

```
WITH view_gaps AS (
  SELECT
    customer_id,
    view_timestamp,
    LEAD(view_timestamp)
      OVER (PARTITION BY customer_id
            ORDER BY view_timestamp)
      AS next_view_time,
    EXTRACT(EPOCH FROM (
      LEAD(view_timestamp)
        OVER (PARTITION BY customer_id
              ORDER BY view_timestamp)
      - view_timestamp
    )) / 60.0 AS gap_minutes
  FROM web_events.page_views
)
SELECT
  customer_id,
  COUNT(*)                        AS view_transitions,
  ROUND(AVG(gap_minutes)::NUMERIC, 2)
                                  AS avg_minutes_between_views
FROM view_gaps
```

Answer (2/2)

✅ ANSWER

```
WHERE gap_minutes IS NOT NULL
  AND gap_minutes < 30
GROUP BY customer_id
ORDER BY avg_minutes_between_views DESC;
```

STORY

The Board Presentation

The CEO's board deck needs three numbers for every month: revenue this month, revenue last month, and the percentage change. For every region. For every product category. These period-over-period comparisons are the bread and butter of business reporting. LAG makes them trivial.

The Period-over-Period Formula

Every period-over-period calculation follows the same pattern:

Period-over-Period Template

```
SELECT
  period,
  metric,
  LAG(metric) OVER (ORDER BY period)        AS prev_period,
  metric - LAG(metric) OVER (ORDER BY period)
                                             AS absolute_change,
  ROUND(
    (metric - LAG(metric) OVER (ORDER BY period))
    / NULLIF(LAG(metric) OVER (ORDER BY period), 0)
    * 100, 2
  )                                          AS pct_change
FROM aggregated_data;
```

Three things to remember:

- Always use NULLIF on the denominator to avoid division by zero
- The first period always shows NULL -- there is no 'previous'
- Add PARTITION BY when you need period-over-period within groups

Month-over-Month by Region

MoM growth for each region independently:

MoM by Region (1/2)

```
WITH monthly AS (
  SELECT
    dr.region_name AS region,
    DATE_TRUNC('month', o.order_date) AS order_month,
    SUM(o.net_total)               AS revenue
  FROM sales.orders o
  JOIN stores.stores s ON o.store_id = s.store_id
  JOIN core.dim_region dr ON s.region_id = dr.region_id
  WHERE o.order_status = 'Delivered'
  GROUP BY dr.region_name, DATE_TRUNC('month', o.order_date)
)
SELECT
  region,
  order_month,
  revenue,
  LAG(revenue) OVER (PARTITION BY region
                     ORDER BY order_month)
                                     AS prev_month,
  ROUND(
    (revenue - LAG(revenue)
```

MoM by Region (2/2)

```
      OVER (PARTITION BY region
            ORDER BY order_month))
    / NULLIF(LAG(revenue)
      OVER (PARTITION BY region
            ORDER BY order_month), 0)
    * 100, 2
  )                                  AS mom_growth_pct
FROM monthly
ORDER BY region, order_month;
```

Year-over-Year Comparison

For YoY, use LAG with offset 12 (assuming monthly data):

YoY Growth (1/2)

```
WITH monthly AS (
  SELECT
    DATE_TRUNC('month', order_date) AS order_month,
    SUM(net_total)                  AS revenue
  FROM sales.orders
  WHERE order_status = 'Delivered'
  GROUP BY DATE_TRUNC('month', order_date)
)
SELECT
  order_month,
  revenue,
  LAG(revenue, 12)
    OVER (ORDER BY order_month)     AS same_month_last_year,
  ROUND(
    (revenue - LAG(revenue, 12)
      OVER (ORDER BY order_month))
    / NULLIF(LAG(revenue, 12)
      OVER (ORDER BY order_month), 0)
    * 100, 2
  )                                 AS yoy_growth_pct
```

YoY Growth (2/2)

```
FROM monthly
ORDER BY order_month;
```

PRO TIP

PRO TIP

LAG(col, 12) only works for YoY when every month has exactly one row. If any month is missing, the offset will grab the wrong month. For bulletproof YoY, join on month number instead of using LAG offset.

Detecting Anomalies with LAG

Flag rows where the value changed by more than a threshold:

Anomaly Detection (1/2)

```
WITH daily_revenue AS (
  SELECT
    order_date::DATE              AS rev_date,
    SUM(net_total)              AS revenue
  FROM sales.orders
  WHERE order_status = 'Delivered'
  GROUP BY order_date::DATE
)
SELECT
  rev_date,
  revenue,
  LAG(revenue) OVER (ORDER BY rev_date)
                                  AS prev_day_revenue,
  ROUND(
    (revenue - LAG(revenue) OVER (ORDER BY rev_date))
    / NULLIF(LAG(revenue) OVER (ORDER BY rev_date), 0)
    * 100, 2
  )                               AS pct_change,
  CASE
    WHEN LAG(revenue) OVER (ORDER BY rev_date) IS NULL
```

Anomaly Detection (2/2)

```
      THEN 'first_day'
    WHEN ABS(revenue - LAG(revenue) OVER (ORDER BY rev_date))
         / NULLIF(LAG(revenue) OVER (ORDER BY rev_date), 0)
         > 0.20
      THEN 'ANOMALY'
    ELSE 'normal'
  END                              AS flag
FROM daily_revenue
ORDER BY rev_date;
```

COMMON MISTAKE

Common LAG/LEAD Mistakes

Forgetting ORDER BY inside OVER: LAG without ORDER BY gives unpredictable results. PostgreSQL may not even error -- it just picks an arbitrary 'previous' row.

Wrong partition boundaries: If you partition by store_id, LAG resets at each store. Miss the partition and you compare Store 5 January to Store 4 December.

Division by zero in growth calc: When previous period revenue is 0, dividing gives an error. Always wrap with NULLIF(prev, 0).

Assuming LAG(col, 12) equals same-month-last-year: It only works if every period has exactly one row. Missing months shift the offset.

Practice 5: Price Change Detection

HARD

SCENARIO: The procurement team suspects some suppliers are quietly raising prices between purchase orders. They want a report flagging significant price jumps.

TASK: For each product, compare each order_item's unit_price to the previous order_item's unit_price (ordered by order_date). Flag any price change greater than 10% as 'PRICE_JUMP'. Show product_id, order_date, unit_price, prev_price, pct_change, and the flag.

HINT: Join order_items to orders for the date. Use LAG on unit_price partitioned by product_id, ordered by order_date. Calculate percentage change and use CASE WHEN for the flag.

Answer (1/2)

✅ ANSWER

```
WITH price_history AS (
  SELECT
    oi.prod_id,
    o.order_date,
    oi.unit_price,
    LAG(oi.unit_price)
      OVER (PARTITION BY oi.prod_id
            ORDER BY o.order_date) AS prev_price
  FROM sales.order_items oi
  JOIN sales.orders o ON oi.order_id = o.order_id
)
SELECT
  prod_id,
  order_date,
  unit_price,
  prev_price,
  ROUND(
    (unit_price - prev_price)
    / NULLIF(prev_price, 0) * 100, 2
  )                                AS pct_change,
  CASE
    WHEN prev_price IS NULL THEN 'first_record'
```

Answer (2/2)

✅ ANSWER

```
    WHEN ABS(unit_price - prev_price)
         / NULLIF(prev_price, 0) > 0.10
      THEN 'PRICE_JUMP'
    ELSE 'stable'
  END                              AS flag
FROM price_history
WHERE prev_price IS NOT NULL
  AND ABS(unit_price - prev_price)
      / NULLIF(prev_price, 0) > 0.10
ORDER BY pct_change DESC;
```

Practice 6: Daily Revenue Anomaly Flagging

HARD

SCENARIO: The ops team wants an automated daily alert for revenue anomalies -- any day where revenue drops more than 20% compared to the previous day.

TASK: Calculate daily delivered revenue. Use LAG to get the previous day's revenue. Flag days where revenue dropped by more than 20% from the previous day. Show rev_date, revenue, prev_day_revenue, pct_change, and an anomaly_flag column.

HINT: Focus on drops only (negative percentage changes greater than 20% in magnitude). Use a CTE for the daily aggregation, then apply LAG in an outer query.

Answer (1/2)

✅ ANSWER

```
WITH daily AS (
  SELECT
    order_date::DATE   AS rev_date,
    SUM(net_total)  AS revenue
  FROM sales.orders
  WHERE order_status = 'Delivered'
  GROUP BY order_date::DATE
),
with_lag AS (
  SELECT
    rev_date,
    revenue,
    LAG(revenue) OVER (ORDER BY rev_date) AS prev_day_revenue
  FROM daily
)
SELECT
  rev_date,
  revenue,
  prev_day_revenue,
  ROUND(
    (revenue - prev_day_revenue)
    / NULLIF(prev_day_revenue, 0) * 100, 2
```

Answer (2/2)

✅ ANSWER

```
  )                   AS pct_change,
  CASE
    WHEN prev_day_revenue IS NULL THEN 'no_comparison'
    WHEN (revenue - prev_day_revenue)
         / NULLIF(prev_day_revenue, 0) < -0.20
      THEN 'DROP_ANOMALY'
    ELSE 'normal'
  END                 AS anomaly_flag
FROM with_lag
ORDER BY rev_date;
```

Practice 7: Complete MoM + YoY Board Report

CRAZY

SCENARIO: The CFO needs a single report for the board meeting that shows monthly revenue with both month-over-month AND year-over-year growth percentages, broken down by region.

TASK: Build a comprehensive report showing: region, month, revenue, previous month revenue, MoM growth %, same month last year revenue, and YoY growth %. Order by region then month. Round all percentages to 2 decimal places.

HINT: Use a CTE for monthly revenue by region. Then apply LAG(revenue, 1) for MoM and LAG(revenue, 12) for YoY, both partitioned by region. Use NULLIF for all division operations.

Answer (1/2)

✅ ANSWER

```
WITH monthly AS (
  SELECT
    dr.region_name AS region,
    DATE_TRUNC('month', o.order_date) AS order_month,
    SUM(o.net_total)               AS revenue
  FROM sales.orders o
  JOIN stores.stores s ON o.store_id = s.store_id
  JOIN core.dim_region dr ON s.region_id = dr.region_id
  WHERE o.order_status = 'Delivered'
  GROUP BY dr.region_name,
           DATE_TRUNC('month', o.order_date)
)
SELECT
  region,
  order_month,
  revenue,
  LAG(revenue, 1)
    OVER w                        AS prev_month_rev,
  ROUND(
    (revenue - LAG(revenue, 1) OVER w)
    / NULLIF(LAG(revenue, 1) OVER w, 0)
    * 100, 2
```

Answer (2/2)

✅ ANSWER

```
  )                              AS mom_growth_pct,
  LAG(revenue, 12)
    OVER w                       AS same_month_last_year,
  ROUND(
    (revenue - LAG(revenue, 12) OVER w)
    / NULLIF(LAG(revenue, 12) OVER w, 0)
    * 100, 2
  )                              AS yoy_growth_pct
FROM monthly
WINDOW w AS (PARTITION BY region
             ORDER BY order_month)
ORDER BY region, order_month;
```

INTERVIEW

Q: What is the difference between LAG and LEAD?

A: LAG looks at previous rows (backward in the window order), LEAD looks at next rows (forward). LAG(col, 1) gets the value from the row before the current one; LEAD(col, 1) gets the value from the row after. They share identical syntax: (column, offset, default). The choice depends on your intent -- LAG for 'change from previous' comparisons, LEAD for 'time until next event' calculations.

INTERVIEW

Q: How do you calculate month-over-month growth in SQL?

A: Aggregate to monthly level, then use LAG to get the previous month's value. The formula is: (current - LAG(current)) / NULLIF(LAG(current), 0) * 100. Always use NULLIF to avoid division by zero on the first month. Add PARTITION BY for per-group MoM (by region, category, etc.).

INTERVIEW

Q: Can you use LAG/LEAD with aggregate functions?

A: Yes. Window functions execute after GROUP BY, so you can write LAG(SUM(amount)) OVER (...). The SUM is calculated first per group, then LAG operates on those aggregated results. This is the standard pattern for period-over-period metrics on grouped data.

HW 1: Employee Salary Change

EASY

SCENARIO: HR wants to track how employee salaries have changed over time.

TASK: From hr.salary_history, show each employee's monthly payment alongside their previous month's payment. Calculate the absolute change. Order by employee_id and payment_date.

HINT: LAG(monthly_salary) partitioned by employee_id, ordered by payment_date.

Answer

✅ ANSWER

```
SELECT
  employee_id,
  payment_date,
  amount,
  LAG(amount)
    OVER (PARTITION BY employee_id
          ORDER BY payment_date)   AS prev_salary,
  amount - LAG(amount)
    OVER (PARTITION BY employee_id
          ORDER BY payment_date)   AS salary_change
FROM hr.salary_history
ORDER BY employee_id, payment_date;
```

HW 2: Consecutive Store Revenue

EASY

SCENARIO: Regional managers want to see each store's monthly revenue trend.

TASK: For each store, calculate monthly revenue and show the previous month's revenue using LAG. Include the difference between current and previous month.

HINT: Group by store_id and month, then apply LAG partitioned by store_id.

Answer

✅ ANSWER

```
SELECT
  store_id,
  DATE_TRUNC('month', order_date)  AS order_month,
  SUM(net_total)                AS revenue,
  LAG(SUM(net_total))
    OVER (PARTITION BY store_id
          ORDER BY DATE_TRUNC('month', order_date))
                                  AS prev_month_rev,
  SUM(net_total) - LAG(SUM(net_total))
    OVER (PARTITION BY store_id
          ORDER BY DATE_TRUNC('month', order_date))
                                  AS revenue_change
FROM sales.orders
WHERE order_status = 'Delivered'
GROUP BY store_id, DATE_TRUNC('month', order_date)
ORDER BY store_id, order_month;
```

HW 3: Customer Order Frequency

MEDIUM

SCENARIO: The retention team needs to understand customer ordering patterns.

TASK: For each customer with 3+ orders, calculate the average number of days between consecutive orders. Show customer_id, total_orders, and avg_days_between_orders (rounded to 1 decimal).

HINT: Use LAG to get previous order date, calculate the gap, then AVG the gaps per customer.

Answer

✅ ANSWER

```
WITH order_gaps AS (
  SELECT
    cust_id,
    order_date,
    order_date - LAG(order_date)
      OVER (PARTITION BY cust_id
            ORDER BY order_date) AS days_gap
  FROM sales.orders
)
SELECT
  cust_id,
  COUNT(*) + 1                  AS total_orders,
  ROUND(AVG(days_gap)::NUMERIC, 1)
                                AS avg_days_between_orders
FROM order_gaps
WHERE days_gap IS NOT NULL
GROUP BY cust_id
HAVING COUNT(*) >= 2
ORDER BY avg_days_between_orders;
```

HW 4: Category Revenue Momentum

MEDIUM

SCENARIO: The merchandising team wants to know which product categories are gaining or losing momentum month over month.

TASK: Calculate MoM revenue growth percentage for each product category. Flag categories as 'accelerating' (positive growth), 'decelerating' (negative growth), or 'stable' (within +/- 5%). Show only the most recent complete month.

HINT: Category isn't on products directly — chain through dim_brand to dim_category. Group by category and month. LAG for previous month. Trap: products.products has no 'category' column; selecting p.category will error — the path is products → dim_brand → dim_category.

Answer (1/3)

✅ ANSWER

```
WITH monthly_cat AS (
  SELECT
    cat.category_name,
    DATE_TRUNC('month', o.order_date) AS order_month,
    SUM(oi.quantity * oi.unit_price)  AS revenue
  FROM sales.order_items oi
  JOIN sales.orders o ON oi.order_id = o.order_id
  JOIN products.products p ON oi.prod_id = p.product_id
  JOIN core.dim_brand b ON p.brand_id = b.brand_id
  JOIN core.dim_category cat ON b.category_id = cat.category_id
  WHERE o.order_status = 'Delivered'
  GROUP BY cat.category_name,
           DATE_TRUNC('month', o.order_date)
),
with_growth AS (
  SELECT
    category_name,
    order_month,
    revenue,
    LAG(revenue)
      OVER (PARTITION BY category_name
            ORDER BY order_month) AS prev_revenue,
```

Answer (2/3)

✅ ANSWER

```
    ROUND(
      (revenue - LAG(revenue)
        OVER (PARTITION BY category_name
              ORDER BY order_month))
      / NULLIF(LAG(revenue)
        OVER (PARTITION BY category_name
              ORDER BY order_month), 0)
      * 100, 2
    )                              AS mom_growth_pct
  FROM monthly_cat
)
SELECT
  category_name,
  order_month,
  revenue,
  prev_revenue,
  mom_growth_pct,
  CASE
    WHEN mom_growth_pct > 5  THEN 'accelerating'
    WHEN mom_growth_pct < -5 THEN 'decelerating'
    ELSE 'stable'
  END AS momentum
```

Answer (3/3)

✅ ANSWER

```
FROM with_growth
WHERE order_month = (
  SELECT MAX(order_month) FROM with_growth
    WHERE prev_revenue IS NOT NULL
)
ORDER BY mom_growth_pct DESC;
```

HW 5: Supply Chain Delivery Trend

HARD

SCENARIO: The logistics director wants to track whether delivery times are improving or worsening month over month.

TASK: Calculate the average delivery time (days from order to shipment delivery) per month. Use LAG to show the previous month's average and the change. Flag months where delivery time increased by more than 1 day as 'SLIPPING'.

HINT: Join orders to shipments. Calculate delivery_days per order. Aggregate by month, then apply LAG.

Answer (1/2)

✅ ANSWER

```
WITH delivery_times AS (
  SELECT
    DATE_TRUNC('month', o.order_date) AS order_month,
    ROUND(AVG(sh.delivered_date - o.order_date)::NUMERIC, 2)
                                      AS avg_delivery_days
  FROM sales.orders o
  JOIN sales.shipments sh ON o.order_id = sh.order_id
  WHERE sh.delivered_date IS NOT NULL
  GROUP BY DATE_TRUNC('month', o.order_date)
)
SELECT
  order_month,
  avg_delivery_days,
  LAG(avg_delivery_days)
    OVER (ORDER BY order_month)   AS prev_month_avg,
  ROUND(
    avg_delivery_days - LAG(avg_delivery_days)
      OVER (ORDER BY order_month), 2
  )                               AS change_in_days,
  CASE
    WHEN LAG(avg_delivery_days)
         OVER (ORDER BY order_month) IS NULL
```

Answer (2/2)

✅ ANSWER

```
      THEN 'no_comparison'
    WHEN avg_delivery_days - LAG(avg_delivery_days)
         OVER (ORDER BY order_month) > 1
      THEN 'SLIPPING'
    WHEN avg_delivery_days - LAG(avg_delivery_days)
         OVER (ORDER BY order_month) < -1
      THEN 'IMPROVING'
    ELSE 'stable'
  END                              AS trend_flag
FROM delivery_times
ORDER BY order_month;
```

HW 6: Multi-Metric Executive Dashboard

CRAZY

SCENARIO: Build a single query that the CEO can use to see monthly performance across revenue, order count, and average order value -- each with MoM change.

TASK: Create a monthly report with: month, revenue, revenue_mom_pct, order_count, orders_mom_pct, avg_order_value, aov_mom_pct. All percentages rounded to 2 decimals. Use a named WINDOW clause to avoid repeating the window definition.

HINT: Aggregate all three metrics in a CTE. Apply LAG three times in the outer query. Use WINDOW w AS (...) and reference it with OVER w.

Answer (1/2)

✅ ANSWER

```
WITH monthly AS (
  SELECT
    DATE_TRUNC('month', order_date) AS order_month,
    SUM(net_total)               AS revenue,
    COUNT(*)                        AS order_count,
    ROUND(AVG(net_total)::NUMERIC, 2)
                                    AS avg_order_value
  FROM sales.orders
  WHERE order_status = 'Delivered'
  GROUP BY DATE_TRUNC('month', order_date)
)
SELECT
  order_month,
  revenue,
  ROUND(
    (revenue - LAG(revenue) OVER w)
    / NULLIF(LAG(revenue) OVER w, 0) * 100, 2
  )                                 AS revenue_mom_pct,
  order_count,
  ROUND(
    (order_count - LAG(order_count) OVER w)::NUMERIC
    / NULLIF(LAG(order_count) OVER w, 0) * 100, 2
```

Answer (2/2)

✅ ANSWER

```
  )                                AS orders_mom_pct,
  avg_order_value,
  ROUND(
    (avg_order_value - LAG(avg_order_value) OVER w)
    / NULLIF(LAG(avg_order_value) OVER w, 0) * 100, 2
  )                                AS aov_mom_pct
FROM monthly
WINDOW w AS (ORDER BY order_month)
ORDER BY order_month;
```

KEY TAKEAWAYS

KEY TAKEAWAYS

LAG looks backward: LAG(col) gets the previous row's value. Use it for 'change from last period' calculations.
LEAD looks forward: LEAD(col) gets the next row's value. Use it for 'time until next event' calculations.
Three arguments: LAG(column, offset, default) -- offset defaults to 1, default defaults to NULL.
NULLIF prevents division by zero: Always wrap the denominator in NULLIF(prev_value, 0) for percentage calculations.
PARTITION BY resets the window: LAG within PARTITION BY region gives you per-region comparisons. Without it, you compare across all rows.
Named WINDOW clause: WINDOW w AS (...) lets you define the window once and reference it with OVER w, reducing repetition.

Window Functions -- Complete Toolkit

With earlier sessions, you now have the full window function arsenal:

Complete Window Function Reference

```
-- Window Functions Part 1: Ranking & Bucketing
ROW_NUMBER()  -- unique sequential number
RANK()        -- with gaps on ties
DENSE_RANK()  -- without gaps on ties
NTILE(n)      -- divide into n buckets

-- Window Functions Part 2: Aggregation & Frames
SUM() OVER    -- running/cumulative totals
AVG() OVER    -- moving averages
COUNT() OVER  -- running counts
ROWS BETWEEN  -- frame boundaries

-- Window Functions Part 3: Row Comparison
LAG(col, n)   -- previous row value
LEAD(col, n)  -- next row value
```

What is Next

Tomorrow we shift from WRITING queries to READING how PostgreSQL EXECUTES them. You will learn to use EXPLAIN and EXPLAIN ANALYZE to understand query plans -- the key to knowing WHY a query is slow.